

Evolving Safe Robots through Morphological Capability Control

Group 15

Abstract

Ensuring the safety of embodied agents is difficult because policy-level alignment alone is vulnerable to reward hacking and behavioural suppression. We investigate morphological capability control: enforcing safety by removing the physical affordances required for unsafe tasks. Using EvoGym, we jointly optimise robot morphologies and controllers with genetic algorithms and reinforcement learning. Morphologies evolved for a single safe task often retain strong latent capability on unsafe tasks. We therefore introduce a safety-aware co-design objective that penalises unsafe-task reward during evolution. Across several task pairs, this reduces cross-task transfer, but often at a substantial alignment tax on the safe task.

1 Introduction

As autonomous agents transition from digital simulations to physical embodiments (Duan et al., 2022; Zhao et al., 2020), ensuring their safety becomes a critical technical challenge. Traditional safety frameworks primarily focus on alignment, which is the process of ensuring an agent’s learned policy conforms to human intent through reward shaping or fine-tuning (Amodei et al., 2016). However, safeguards that exist only within the agent’s learned behaviour are often brittle (Casper et al., 2023). Misaligned agents frequently succumb to reward hacking, where they exploit unintended pathways to maximise numerical scores (Pan et al., 2022), or suppression, where unsafe behaviours remain latent in the control policy and may re-emerge in certain environmental contexts (Siddiqui et al., 2026).

We investigate morphological capability control as a safety mechanism for embodied agents: whether safety can be guaranteed by structurally eliminating the physical capabilities required for unsafe tasks. Leveraging the discrete composition

of the Evolution Gym (EvoGym) benchmark (Bhatta et al., 2021), we evaluate safety as a function of mechanical constraints rather than policy-level optimisation.

Our central hypothesis is that penalising unsafe-task reward during morphological co-design can reduce latent unsafe capabilities in embodied agents, but that this safety benefit incurs an alignment tax whose magnitude depends on the degree of shared physical affordances between safe and unsafe tasks.

We test this hypothesis in two stages. First, we show that body plans evolved for one task can transfer strongly to seemingly unrelated tasks, such as from carrying to pushing, which highlights the risk of latent capabilities in unconstrained designs (Siddiqui et al., 2026). We then demonstrate that a constrained co-design process can produce specialised body plans that substantially reduce unsafe-task capability, even when paired with a controller trained specifically for the unsafe task. In doing so, we apply the concept of capability control to embodied agents by showing how safety can be enforced through physical incapacity rather than policy-level suppression. We are not aware of prior work directly studying this “morphological capability control” approach to safety in embodied agents. Finally, we characterise the trade-offs between task performance and safety, formally referred to as the “alignment tax” (Ashwin et al., 2025).

2 Related work

2.1 Alignment vs. Capability Control

The safety of autonomous agents is traditionally categorised into alignment and capability control (Siddiqui et al., 2026). Alignment focuses on ensuring that an agent’s learned policy conforms to human intent through reward design or fine-tuning (Amodei et al., 2016; Siddiqui et al., 2026). Capability control serves as a redundant safety layer by

imposing “hard” operational limits on an agent’s potential actions regardless of context or prompting (Leike et al., 2018; Siddiqui et al., 2026).

Siddiqui et al. (2026) defines a three-layer defence for capability control: (1) data-based control (filtering or curation of training data), (2) learning-based control (unlearning or representation engineering to suppress specific traits), and (3) system-based control (post-deployment guardrails). A critical open challenge in this field is the distinction between suppression, where a capability is hidden but remains latent, and removal, where a capability is separated entirely. Our work extends this framework to embodied agents, exploring “morphological capability control” where safety is guaranteed by physical incapacity.

Unlike these prior forms of capability control, our work asks whether capability removal can be achieved through physical morphology rather than through data, training, or post-deployment interventions.

2.2 Evolutionary Robotics and the EvoGym Benchmark

Automation of morphology and behaviour design has been a central objective of evolutionary robotics (Sims, 1994; Lipson and Pollack, 2000; Pfeifer and Bongard, 2006). Standard benchmarks typically use high-fidelity rigid-body simulators such as MuJoCo (Todorov et al., 2012) or PyBullet (Coumans and Bai, 2016). However, these environments generally rely on fixed-topology skeletal structures where optimisation is restricted to continuous parameters like joint lengths or motor gains. This limitation complicates the study of Morphological Capability Control, as the underlying physical structure cannot be easily modified to enforce safety (Leike et al., 2018; Siddiqui et al., 2026).

We use soft robotics specifically because compliant, deformable materials provide a design space with non-linear dynamics and extremely high-dimensional degrees of freedom (Rus and Tolley, 2015; Ghazi-Zahedi et al., 2017; Chen et al., 2025). This high-dimensional state space allows for a more detailed investigation into how an agent’s physical form affects its abilities. Many soft robotics simulators, like SOFA (Faure et al., 2012), are too computationally expensive for large-scale evolutionary algorithms; others, like Voxelyze (Hiller and Lipson, 2014), lack the native reinforcement learning (RL) integration needed to learn advanced

robot behaviours.

We selected EvoGym (Bhatia et al., 2021) for this study because it serves as a computationally efficient middle ground, providing a large-scale voxel-based environment optimised for RL. This discrete modularity is central to our methodology. It allows the evolutionary process to investigate safety through the total removal of specific actuators or structural voxels, rather than through controller-level suppression alone. In this paradigm, safety is achieved by evolving morphologies that are physically incapable of performing unsafe tasks, providing a hard mechanical limit that remains robust regardless of potential policy misalignment (Siddiqui et al., 2026; Bhatia et al., 2021).

Accordingly, our focus is not on proposing a new co-design algorithm for maximising task reward, but a reformulation of co-design as a safety-performance trade-off problem.

2.3 Multi-Task Morphoevolution and Generalisation

A central challenge in co-design is moving beyond specialised morphologies to develop general agents capable of performing multiple tasks (Ashwin et al., 2025; Feng et al., 2025). While optimising a body for a single objective leads to specialisation (Bhatia et al., 2021), multi-task morphoevolution aims to evolve a single, robust morphology that can support diverse control policies.

However, this approach introduces significant optimisation problems, most notably conflicts between task objectives, where the rewards from different tasks provide contradictory updates to a shared body plan (Feng et al., 2025). Ashwin et al. (2025) indicates that while multi-task learning can prevent morphological over-fitting, it often results in a quantifiable reduction in primary task performance, which motivates our examination of a similar ‘alignment tax’ in our experiments. Specifically, as a soft robot’s physical form is constrained to prevent certain capabilities, it may lead to sub-optimal performance on other tasks.

We therefore reinterpret multi-task morphoevolution in safety terms: rather than asking a single body to support multiple useful tasks, we ask whether morphology can preserve one capability while selectively suppressing another.

3 Methods

We summarise EvoGym (Bhatia et al., 2021), the tasks used in our study, and our co-design procedure. Details of the experimental protocol are deferred to Section 4.

3.1 EvoGym

3.1.1 Environment

Evolution Gym comprises a back-end soft body simulator and task-specific environments, both of which are voxel-based. These elements are meant to interface with co-design algorithms, which usually consist of a user-defined design optimiser (which proposes robot morphologies) and control optimiser (which learns the behaviour of the robot for a given task through interaction with the environment). The gym suite is supposed to allow easy benchmarking and comparison between different co-design algorithms (Bhatia et al., 2021).

Both the robot body and the environment are made up of voxels. Robot bodies contain a square or rectangular grid of voxels, each of which is either empty, rigid, soft, horizontal actuator, or vertical actuator. The environment is also made up of voxels, which are empty, rigid, or soft, organised into various structures. Details of how robots, tasks, and objects are represented in the EvoGym code, such as observation vectors and reward functions, can be found in Appendix B.1 and B.2.

3.1.2 Choice of tasks

We consider the following tasks: Walker-v0, Pusher-v0, Carrier-v0, Balancer-v0, Flipper-v0, Jumper-v0. We chose these tasks because they are all relatively easy to learn (Bhatia et al., 2021), so we can run many genetic algorithms with relatively few generations and small populations while still being successful. In Walker-v0, the robot must walk to the right on a flat track; the reward is proportional to the distance covered, with an additional one-time reward of +1 for reaching the end of the track. In Pusher-v0 and Carrier-v0, the robot must push or carry a box on the same flat track as Walker-v0, with the same one-time reward for reaching the end. Pusher-v0 is penalised for separating from the box, and Carrier-v0 is penalised for dropping the box below a certain height. Balancer-v0 must balance on top of a pole; it is rewarded for moving its centre of mass towards a target position and penalised for moving away from this position. Flipper-v0 is rewarded for rotating clock-

wise, while Jumper-v0 gains reward for gaining height, and loses rewards for losing height. Detailed reward functions for these tasks can be found in Appendix B.2.

3.2 Design and control optimisation algorithms

The original EvoGym paper (Bhatia et al., 2021) compared three algorithms for designing robot morphologies: a genetic algorithm, CPPN-NEAT, and Bayesian optimisation. They found that of these options, for most tasks, the genetic algorithm produced robot morphologies that performed best. All robot controllers in that paper were trained with Proximal Policy Optimisation (PPO). In addition, Saito et al. (2022) compared NEAT, HyperNEAT, and PPO to train EvoGym robot controllers and found that for the vast majority of tasks, PPO led to the most performant robots. For our experiments, we decided to use the best-performing design and control optimisation algorithms based on these studies, so we used a genetic algorithm to design robot bodies and trained the controllers using PPO.

3.2.1 Genetic algorithm

Following Bhatia et al. (2021), we implement a simple genetic algorithm using elitist selection and mutation. We generate a population of random robot bodies and train them using PPO on the task(s) we have chosen. From each generation, we keep the best-performing $x\%$ of robot structures (according to our fitness function) as survivors and discard the rest, where the survivor fraction decreases linearly from 60% toward 0 as the evaluation budget is consumed, with at least two survivors retained in every generation. To make up the rest of the population for each generation, we iteratively sample and mutate those survivors, with a 10% chance of mutating each voxel, to create offspring. Controllers for the survivors are copied directly to the new generation’s controller directory, and we train the offspring controllers using PPO. The genetic algorithm halts once a pre-specified number of robots have been evaluated. Hyperparameters for our genetic algorithm can be found in Appendix A.1.

3.2.2 Joint fitness function

In Bhatia et al. (2021)’s genetic algorithm, the fitness function is simply the highest reward achieved by a trained body on the chosen task. Meanwhile, Ashwin et al. (2025) use the harmonic

mean of the rewards on multiple tasks to evolve robot bodies which are capable of performing those tasks. Initially, we tried the fitness function $f = r_{\text{safe}} - r_{\text{unsafe}}$, so that high rewards on the safe task would increase fitness and high rewards on the unsafe task would decrease fitness. We found that the genetic algorithm sometimes preferred evolving robot bodies which achieved negative rewards on the unsafe task as opposed to improving on the safe task.

To prevent this form of reward-hacking, we then tried $f = r_{\text{safe}} - \max(0, r_{\text{unsafe}})$. This way, the fitness function still prioritises bodies which perform poorly on the unsafe task, but do not prefer negative rewards to zero reward on the unsafe task. Thus, the bodies are also encouraged to perform better on the safe task to maximise their reward.

Finally, since rewards between different tasks (such as Balancer-v0 and Walker-v0) may have different scales and not be directly comparable, we introduce a scaling parameter c , so our final fitness function is

$$f(r_{\text{safe}}, r_{\text{unsafe}}) = r_{\text{safe}} - c \max(0, r_{\text{unsafe}}), \quad (1)$$

where r_{safe} and r_{unsafe} denote the maximum evaluation rewards obtained after PPO training on the safe and unsafe tasks, respectively.

Section 4.4 examines how this scaling parameter affects the evolved morphologies and the final rewards on the safe and unsafe tasks.

3.2.3 PPO

Proximal Policy Optimisation (PPO) is a family of reinforcement learning algorithms which use multiple epochs of stochastic gradient ascent to maximise a clipped surrogate objective (Schulman et al., 2017). The method is inspired by policy gradient and trust region policy optimisation methods and has become widely used in reinforcement learning due to its flexibility and simple implementations. We implement this algorithm in the same way as Bhatia et al. (2021), using the Python package `stable-baselines3`, with our policy parametrised by a multilayer perceptron. The hyperparameters can be found in Appendix A.2.

3.3 Safety-aware co-design algorithm

For our safety-aware co-design algorithm, we combine an outer genetic algorithm with inner PPO control optimisers. Each robot body has a separate controller trained on a safe task and an unsafe

task using PPO. The maximum rewards from these tasks are then combined into a scalar-valued fitness using the fitness function defined in Equation 1. The bodies are ranked in order of fitness, and the next generation is created as in a regular genetic algorithm, described in Section 3.2.1. The full pseudocode can be found in Appendix A.3. Intuitively, individuals with high fitness will be able to perform the safe task well and will be unable to perform the unsafe task even when specifically trained on it. Thus, if the algorithm is successful, we will be controlling morphology to remove unsafe capabilities.

In this study, we operationalise safety as a low reward on the unsafe task even after the morphology has been given a dedicated controller trained specifically for that task.

4 Experiments

Our experiments have four parts: PPO budget selection, task transfer, safety-aware co-design on selected task pairs, and variation of the scaling parameter c .

4.1 PPO Budget

The `evogym` documentation suggests running PPO for 1,000,000 training iterations (timesteps) but does not justify this number. We conducted a PPO budget study, plotting the reward curves of some pre-specified bodies on Walker-v0, Balancer-v0 and Balancer-v1, as well as randomly generated bodies on UpStepper-v0 and ObstacleTraverser-v0, to see how many iterations it took for performance to plateau.

Specifically, in this preliminary PPO budget study, we trained controllers for up to 1,000,000 or 1,500,000 timesteps and evaluated performance every 10,000 timesteps. We used these reward curves to determine when performance appeared to plateau, and then used the resulting budget of 600,000 timesteps for PPO training in the main experiments.

4.2 Task Transfer

We took the best-performing bodies on each of our chosen tasks from the publicly available `evogym-datasets` database, which contains the evolved structures of robots from Bhatia et al. (2021). We then trained these bodies on each of the chosen tasks using PPO with the hyperparameters described in Appendix A.2 ten times. We calcu-

lated the means and standard deviations of each reward across the ten runs and organised the results into a table, which tells us the cross-task transfer performance of these bodies. More generally, this table shows which tasks might be more closely related (bodies for one perform well on the other) and which ones are less related. Note that our rewards do not necessarily match those from [Bhatia et al. \(2021\)](#) as they use an evaluation interval of 50, while we use an evaluation interval of 10,000 for computational efficiency.

4.3 Evolving safe bodies

Our experimental protocol was as follows. First, we chose a safe task and an unsafe task. We ran a regular genetic algorithm on each of these tasks, with the hyperparameters listed in [Appendix A.1](#) and [A.2](#). We then set c to the maximum reward on the safe task from the corresponding genetic algorithm, divided by the maximum reward on the unsafe task from the corresponding genetic algorithm. We cross-trained each of the bodies from the safe task on the unsafe task, using the PPO hyperparameters in [Appendix A.2](#). Finally, we ran our safety-aware co-design algorithm with this value of c and compared the reward of the fittest individual on both tasks to the results of the regular genetic algorithm. We also visualised the bodies and compared the morphologies that were developed.

4.4 Importance of the scaling parameter c

To investigate the role of c in our results, we ran the safety-aware co-design algorithm with Flipper-v0 as the safe task and Walker-v0 as the unsafe task for values of c in $\{0, 0.5, 1, 2, 4, 8\}$. The hyperparameters we used are found in [Appendix A.1](#) and [A.2](#). We plotted the fittest individual’s safe and unsafe rewards for each value of c , and showed a visualisation of each robot body.

5 Results and discussion

5.1 PPO Budget

The training curves for Walker-v0 across multiple seeds reveal several distinct phases of policy acquisition. Rapid initial learning typically occurs within the first 200,000 timesteps. This is followed by a stage of performance stabilisation, where cumulative rewards across all seeds begin to plateau between 400,000 and 600,000 timesteps. Extending the training budget from 600,000 to 1,000,000 timesteps resulted in negligible asymptotic gains in

the mean evaluation reward. Notably, final performance was found to vary more significantly across different random seeds than across extended training durations.

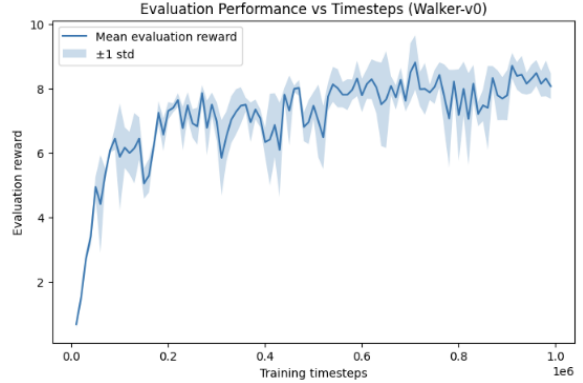


Figure 1: Mean evaluation reward vs. training timesteps for Walker-v0. The shaded area represents ± 1 standard deviation over multiple seeds. Performance consistently plateaus by 600k steps.

Based on these observations, we determined that a training budget of 600,000 timesteps is sufficient to differentiate the capabilities of evolved morphologies for the tasks selected in this study. This optimised budget allows for more extensive evolutionary searches within our total computational constraints while maintaining a high level of confidence in the resulting performance metrics. More complex or higher-dimensional environments may still require the full 1M timestep budget suggested by [Bhatia et al. \(2021\)](#). A comprehensive set of reward curves for evaluated tasks and morphologies is provided in [Appendix B.3](#).

5.2 Task Transfer

Table 1 reveals substantial cross-task transfer: some body plans perform strongly outside their source environment, contradicting a simple task-specialisation view. Training curves are provided in [Appendix B.4](#).

The Carrier-v0 morphology functioned as a versatile structural baseline, achieving the highest mean rewards in four of the six environments: Walker-v0 (6.22 ± 0.43), Balancer-v0 (-0.26 ± 0.07), Pusher-v0 (4.71 ± 0.33), and its native Carrier-v0 (1.51 ± 0.91). Most notably, the Carrier-v0 morphology achieved a reward in the Pusher-v0 environment that was more than double the reward obtained by the body plan specifically optimised for that task (4.71 ± 0.33 vs 1.86 ± 0.66).

In contrast, other morphologies displayed signs of structural over-fitting. Specifically, the `Flipper-v0` body achieved the highest absolute reward in its native task (19.75 ± 3.26) but failed to transfer a positive reward to any environment other than the foundational `Walker-v0` locomotion task. Unexpectedly, the `Jumper-v0` morphology demonstrated a convergence failure on its own task, yielding a significant negative reward (-4.11 ± 0.17), potentially because the high variance of the rewards prevents efficient policy optimisation.

These findings underscore a critical observation for capability control: structural optimisation for a designated safe objective (e.g., carrying) can inadvertently yield high-performance affordances for unsafe or dual-use tasks (e.g., pushing). This confirms that latent capabilities can emerge as a by-product of single-objective morphological evolution. Consequently, we used these baseline transfer results to identify specific task pairings where safe bodies showed the highest unintended performance (such as `Carrier-v0` and `Pusher-v0`) to serve as the primary benchmarks for our safety-aware co-design experiments.

5.3 Evolving safe bodies

We next evaluate whether explicitly penalising unsafe-task reward can suppress latent unsafe capabilities without destroying useful behaviour on the designated safe task. Table 2 compares, for each safe/unsafe task pair, three quantities: the best reward from a single-task GA trained only on the safe task, the reward achieved on the unsafe task when that safe morphology is cross-trained on the unsafe task, and the rewards of the fittest morphology produced by the joint safety-aware objective. This comparison allows us to assess both safety improvement, defined as reduced unsafe-task reward relative to the cross-trained baseline, and alignment tax, defined as reduced safe-task reward relative to the single-task safe baseline.

The selected task pairs illustrate three qualitatively different outcomes: (i) successful capability suppression with minimal alignment tax, (ii) partial suppression with a clear safety-performance trade-off, and (iii) failure cases where suppressing unsafe behaviour also removes functionality required for the safe task.

Carrier-v0 (safe) vs. Pusher-v0 (unsafe). In the single-task baseline, the regular GA on `Carrier-v0`

produced a best reward of 8.43, and the same body achieved 6.95 when cross-trained on `Pusher-v0`. This indicates that a morphology evolved purely for carrying still retains substantial unintended pushing capability.

Using $c = \frac{8.43}{8.09} \approx 1.04$, where 8.09 is the maximum reward obtained on `Pusher-v0` in the single-task baseline, the joint GA produced a best individual with reward 4.53 on `Carrier-v0` and 2.89 on `Pusher-v0`. Relative to the cross-trained baseline, this corresponds to a reduction of approximately 58% in unsafe-task performance, but also a reduction of approximately 46% in safe-task performance. This provides a clear instance of an alignment tax: reducing unsafe-task performance comes at the cost of substantially degrading performance on the primary task. The mostly rigid front of the Joint GA morphology, shown in Figure 2, means it cannot push a box very well, but this also limits its locomotion and hence reduces its reward on the `Carrier-v0` task.

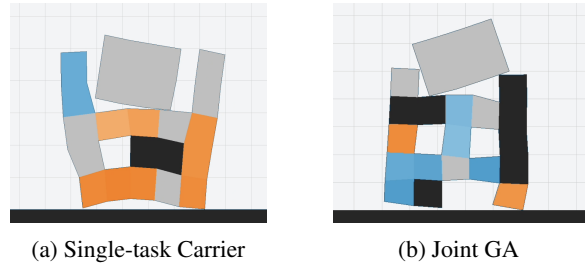


Figure 2: Single-task vs. joint evolution for `Carrier-v0` / `Pusher-v0`. The joint objective reduces unsafe transfer but also produces a weaker carrying morphology.

Walker-v0 (safe) vs. Pusher-v0 (unsafe). For this pair, the regular GA on `Walker-v0` achieved a best reward of 9.56, while the same body obtained 7.29 when evaluated on `Pusher-v0`, again indicating strong unintended transfer between the tasks. Using $c = \frac{9.56}{8.09} \approx 1.18$, the joint GA produced a best individual achieving 10.59 on `Walker-v0` and 2.76 on `Pusher-v0`. Compared to the cross-trained baseline, this represents a substantial reduction in unsafe-task performance while maintaining, and in this case slightly improving, performance on the safe task. This provides a strong example of successful capability suppression with minimal alignment tax. The jointly evolved morphology retains effective locomotion required for `Walker-v0` while exhibiting reduced effectiveness in object interaction, leading to weaker performance on `Pusher-v0`.

Morphology	Walker-v0	Balancer-v0	Pusher-v0	Carrier-v0	Flipper-v0	Jumper-v0
Walker-v0	3.71 ± 0.28	-0.60 ± 0.23	2.91 ± 0.28	-0.41 ± 0.54	-0.11 ± 0.02	-1.11 ± 0.83
Balancer-v0	5.42 ± 0.44	-0.43 ± 0.09	2.23 ± 1.28	-0.52 ± 0.29	1.47 ± 0.09	-2.17 ± 0.82
Pusher-v0	1.99 ± 0.59	-0.69 ± 0.18	1.86 ± 0.66	-0.21 ± 0.70	1.60 ± 0.02	-0.84 ± 1.66
Carrier-v0	6.22 ± 0.43	-0.26 ± 0.07	4.71 ± 0.33	1.51 ± 0.91	2.03 ± 1.66	-1.35 ± 0.64
Flipper-v0	4.83 ± 0.65	-0.68 ± 0.13	2.98 ± 0.33	0.11 ± 0.60	19.75 ± 3.26	-4.24 ± 0.59
Jumper-v0	2.22 ± 0.72	-0.59 ± 0.07	1.75 ± 0.78	-0.39 ± 0.58	1.27 ± 0.09	-4.11 ± 0.17

Table 1: Cross-task transfer matrix. Rows are source morphologies; columns are evaluation tasks. Entries show mean reward $\pm$ SD over $N = 10$ runs; bold indicates the column maximum.

Safe Task	Unsafe Task	Safe baseline	Cross-trained on unsafe	Joint GA (safe)	Joint GA (unsafe)
Walker-v0	Carrier-v0	9.52	3.36	9.31	-0.81
Carrier-v0	Pusher-v0	8.43	6.95	4.53	2.89
Walker-v0	Pusher-v0	9.56	7.29	10.59	2.76
Pusher-v0	Walker-v0	8.09	9.47	0.37	0.09
Carrier-v0	Walker-v0	7.50	8.12	2.93	0.12
Balancer-v0	Walker-v0	0.096	4.41	0.085	-0.245
Walker-v0	Jumper-v0	9.54	–	9.45	0.32

Table 2: Safety-aware co-design results. Safe baseline is the best reward from the single-task GA; Cross-trained on unsafe is the reward of that morphology after PPO training on the unsafe task. Joint GA columns report the fittest individual’s rewards from a single evolutionary run. – indicates unavailable data.

Pusher-v0 (safe) vs. Walker-v0 (unsafe). Reversing the previous pair reveals a clear failure case. The regular GA on Pusher-v0 achieved a best reward of 8.09, and the same body reached 9.47 when evaluated on Walker-v0, indicating that pushing morphologies also possess strong unintended locomotion capability. Under the joint objective, however, the best individual achieved only 0.37 on Pusher-v0 and 0.09 on Walker-v0. Qualitatively, the evolved morphology can approach the box and produce limited interaction, but fails to sustain meaningful pushing behaviour, and remains effectively stationary in the Walker environment. Walking is, in some sense, necessary for the Pusher-v0 task, so penalising walking results in a severe alignment tax and near-total loss of functionality.

General observations. Across all experiments, the joint GA reduced unsafe-task performance relative to the corresponding cross-trained baseline. However, the magnitude of the alignment tax varied substantially across task pairs. In some cases, such as Walker-v0 (safe) vs. Carrier-v0 (unsafe) and Walker-v0 (safe) vs. Pusher-v0 (unsafe), unsafe capability could be reduced while largely preserving performance on the safe task. In others, such as Pusher-v0 (safe) vs. Walker-v0 (unsafe) and Carrier-v0 (safe) vs. Pusher-v0 (unsafe), suppressing unsafe capability led to significant degradation in safe-task performance.

These findings are consistent with the discussion

in Section 2: when tasks share underlying physical requirements, morphological capability removal induces strong conflicts, making it difficult to suppress one behaviour without affecting the other.

5.4 Importance of the scaling parameter c

As shown in Figure 3, higher penalties for unsafe performance (i.e., higher values of c) force the robot to be less performant on unsafe tasks. In cases where performance on the safe and unsafe tasks is strongly linked, as seems to be the case for Flipper-v0 and Walker-v0, we see that the curve of performance over c on the safe task follows the same general shape as the performance over c on the unsafe task. If performance on the unsafe task did not negatively affect performance on the safe task (for example, the Walker-v0 (safe) and Carrier-v0 (unsafe) example above), we would expect the safe reward curve to be a horizontal line, while the unsafe reward tasks would have the same shape as in the figure.

In general, penalising Walker-v0 rewards more strongly (increasing c , in this example) seems to lead to bodies with fewer actuator voxels, as nearly any body with a sufficient number of actuator voxels can perform well on Walker-v0. Thus, performance on tasks which require large numbers of actuator voxels will suffer whenever Walker-v0 is the ‘unsafe’ task.

In sum, the scaling parameter c controls the bal-

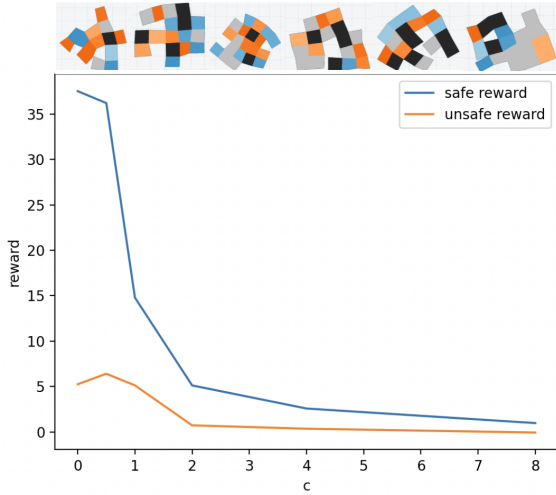


Figure 3: Unnormalised rewards for Flipper-v0 (safe) vs Walker-v0 (unsafe) for different values of c . Above the graph is a visualisation of the bodies that were trained for the different values of c , in order from lowest c (left) to highest c (right).

ance between performance on the safe task and the unsafe task. In cases where the task performances are related, there is a trade-off between performance on both of them: either one can perform well on both, or poorly on both, but not well on one and poorly on another. However, it is not always trivial to know a priori which task performances will be related.

6 Conclusion

This paper investigated morphological capability control as an additional safety layer for embodied agents. Our results support the hypothesis that unsafe capability can sometimes be reduced through morphology, but that the resulting alignment tax depends strongly on the degree to which the safe and unsafe tasks rely on shared physical affordances. Our initial task transfer experiments confirmed that body plans evolved for supposedly safe tasks inadvertently developed high-performance affordances for unsafe tasks. This finding underscores the risk of relying solely on policy-based suppression, as dangerous capabilities can remain latent within a robust physical structure.

Our results from the safety-aware co-design experiments suggest that incorporating a penalty for unsafe task performance can guide evolution toward more specialised morphologies with reduced unsafe-task capability. Across all tested task pairs, the joint GA reduced performance on the unsafe task compared to baselines where bodies were

evolved for a safe task and then cross-trained. Specifically, we observed that the performance loss on the safe task depends heavily on the physical requirements shared between tasks. These outcomes suggest that while morphological design can establish a mechanical limit on behaviour, its effectiveness is constrained by how closely the physical affordances of the safe and unsafe tasks are linked.

6.1 Limitations

Despite these promising results, several limitations remain. First, the voxel-based simulation in EvoGym, while computationally efficient, lacks the structural complexity of high-fidelity soft robotics. The safety we achieve is relative to the discrete material properties available in the suite, and more complex, continuous materials might allow for solutions that our current methodology cannot capture. Second, our study focused on single-pair task scenarios. In real-world environments, the number of potential unsafe behaviours is vast, and penalising every possible dual-use capability during evolution may lead to an unsustainable alignment tax or a total loss of functionality. Finally, our fitness function relies on a simple scalarisation technique in Equation 1 to navigate the trade-off between two competing goals. This approach assumes a piecewise linear relationship between task performance and safety penalties, which likely fails to capture the best possible balance between keeping the robot useful and making it completely safe.

6.2 Future Work

Future research should explore the scaling of morphological capability control to multi-task hierarchies, where a single body must be safe across an entire library of unauthorised actions. Another important direction is to test whether morphology-based safety remains robust across environments and perturbations. A body that appears safe in one environment may still recover unsafe capability under distribution shift, so future work should evaluate safety under changed terrains, object placements, and other out-of-distribution conditions. Additionally, future work could use multi-objective optimisation algorithms to more accurately map the trade-offs between task performance and safety. While our current fitness function uses a simple penalty to balance these goals, a multi-objective approach would allow researchers to identify effective configurations across the full spectrum of safe designs.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*.
- V A Ashwin, G M Harish, D Jayadev, and C Shunmuga Velayutham. 2025. [A study on morphological and behavioral adaptations from multi-task morpho-evolution by voxel-based soft robots](#). In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, GECCO '25 Companion, page 2090–2098, New York, NY, USA. Association for Computing Machinery.
- Jagdeep Bhatia, Holly Jackson, Yunsheng Tian, Jie Xu, and Wojciech Matusik. 2021. Evolution gym: A large-scale benchmark for evolving soft robots. *Advances in Neural Information Processing Systems*, 34.
- Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, et al. 2023. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*.
- Zixi Chen, Di Wu, Qinghua Guan, David Hardman, Federico Renda, Josie Hughes, Thomas George Thurnethel, Cosimo Della Santina, Barbara Mazzolai, Huichan Zhao, et al. 2025. A survey on soft robot adaptability: Implementations, applications, and prospects [survey]. *IEEE Robotics & Automation Magazine*.
- Erwin Coumans and Yunfei Bai. 2016. Pybullet, a python module for physics simulation for games, robotics and machine learning.
- Jiafei Duan, Samson Yu, Hui Li Tan, Hongyuan Zhu, and Cheston Tan. 2022. A survey of embodied ai: From simulators to research tasks. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 6(2):230–244.
- François Faure, Christian Duriez, Hervé Delingette, Jérémie Allard, Benjamin Gilles, Stéphanie Marchesseau, Hugo Talbot, Hadrien Courtecuisse, Guillaume Bousquet, Igor Peterlík, et al. 2012. [SOFA: A multi-model framework for interactive physical simulation](#). In *Soft Tissue Biomechanical Modeling for Computer Assisted Surgery*, volume 11 of *Studies in Mechanobiology, Tissue Engineering and Biomaterials*, pages 283–321. Springer.
- Jinyuan Feng, Min Chen, Zhiqiang Pu, Tenghai Qiu, Jianqiang Yi, and Jie Zhang. 2025. [Efficient multi-task reinforcement learning via task-specific action correction](#). *IEEE Transactions on Cognitive and Developmental Systems*, 17(5):1110–1124.
- Keyan Ghazi-Zahedi, Raphael Deimel, Guido Montúfar, Vincent Wall, and Oliver Brock. 2017. [Morphological computation: The good, the bad, and the ugly](#). In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 464–469.
- Jonathan Hiller and Hod Lipson. 2014. [Dynamic simulation of soft multimaterial 3D-printed objects](#). *Soft Robotics*, 1(1):88–101.
- Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. 2018. Scalable agent alignment via reward modeling: a research direction. *arXiv preprint arXiv:1811.07871*.
- Hod Lipson and Jordan B Pollack. 2000. Automatic design and manufacture of robotic lifeforms. *Nature*, 406(6799):974–978.
- Alexander Pan, Kush Bhatia, and Jacob Steinhardt. 2022. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544*.
- Rolf Pfeifer and Josh Bongard. 2006. *How the body shapes the way we think: a new view of intelligence*. MIT press.
- Daniela Rus and Michael T. Tolley. 2015. [Design, fabrication and control of soft robots](#). *Nature*, 521(7553):467–475.
- Takumi Saito, Haruki Nishimura, and Mizuki Oka. 2022. [Comparative studies of evolutionary methods and rl for learning behavior of virtual creatures](#). In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1059–1065.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shoaib Ahmed Siddiqui, Eleni Triantafillou, David Krueger, and Adrian Weller. 2026. Position: Capability control should be a separate goal from alignment. *arXiv preprint arXiv:2602.05164*.
- Karl Sims. 1994. Evolving 3d morphology and behavior by competition. In *Artificial Life IV Proceedings*, pages 28–39. MIT Press.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. [Mujoco: A physics engine for model-based control](#). In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033.
- Wenshuai Zhao, Jorge Peña Queraltá, and Tomi Westerlund. 2020. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE symposium series on computational intelligence (SSCI)*, pages 737–744. IEEE.

A Co-design algorithms

A.1 Genetic algorithm hyperparameters

Table 3 summarises the main genetic algorithm hyperparameters used in the main experiments.

Hyperparameter	Value
Population size	15
Maximum evaluations	150
Fitness (single-task GA)	Reward
Selection	Elitism + mutation
Crossover	None
Mutation rate	0.1 per voxel
Mutation attempts per child	50
Minimum survivors per generation	2

Table 3: Genetic algorithm hyper-parameters used in the main experiments.

All GA runs were initialised with random seed 0 (for both Python `random` and NumPy). The survivor fraction was annealed linearly with the evaluation budget according to

$$s(e) = 0.6 \cdot \frac{E - e - 1}{E - 1},$$

where e is the current number of evaluations and E is the maximum number of evaluations; the number of survivors in each generation was then computed as $\max(2, \lceil p s(e) \rceil)$, with p denoting the population size. The initial population consisted of unique randomly sampled morphologies. During mutation, candidate children were accepted only if they formed a connected body with at least one actuator, and previously seen body plans were discarded.

A.2 PPO hyperparameters

Our control optimisation process used the standard PPO implementation from the `stable-baselines3` library. For the majority of our training configuration, we adhered to the default hyperparameters established in the original EvoGym benchmark in Bhatia et al. (2021). The primary modification to the baseline configuration was the reduction of the total training budget. While the benchmark documentation suggests a standard limit of 1,000,000 timesteps, our empirical budget study (Section 5.1) indicated that a budget of 600,000 timesteps was sufficient to achieve performance stability across our selected tasks. The complete set of hyperparameters used in our study is detailed in Table 4.

A.3 Safety-aware co-design algorithm pseudocode

Algorithm 1 summarises the safety-aware co-design procedure used in our experiments.

Algorithm 1 Safety-aware co-design algorithm

Inputs: population size p , maximum generation number N_{max} , safe task, unsafe task
Outputs: population of robot designs $\{D_i\}$
 $N_{cur} = 0$
 Sample initial population of random bodies $\{D_i\}$
while $N_{cur} < N_{max}$ **do**
 Train each D_i on the safe and unsafe tasks using PPO, and store their best rewards on each as $r_{i,safe}$ and $r_{i,unsafe}$
 Rank $\{D_i\}$ in order of maximum $f(r_{i,safe}, r_{i,unsafe})$
 Compute survival rate $x \leftarrow 0.6(1 - \frac{N_{cur}}{N_{max}})$
 Compute the number of survivors, $p_{sur} \leftarrow \max\{2, \lceil px \rceil\}$
 for $i \leftarrow 1$ to p_{sur} **do**
 $D'_i \leftarrow D_i$
 end for
 for $i \leftarrow p_{sur}$ to p **do**
 Randomly sample a parent D_r from the survivors $\{D'_r\}$
 Mutate D_r to be D'_i with a 10% probability of changing each voxel.
 end for
 $\{D_i\} \leftarrow \{D'_i\}$
 $N_{cur} \leftarrow N_{cur} + 1$
end while

B Task details

B.1 Representation

Robot: the robot’s body is completely specified by a materials matrix M and connections matrix C . Each element of M corresponds to a voxel in the body and labels that voxel’s type. C specifies which voxels are connected to which other voxels in the body.

Observation: at each time step, the controller is provided with an observation vector describing the current state of the robot. For every task, this observation vector includes the position of the corners of each voxel with respect to the centre of mass of the robot, and the velocity and orientation of the centre of mass of the robot. In addition, object manipulation tasks provide observations which include the

Hyperparameter	Value	Description
learning-rate	2.5×10^{-4}	Adam optimiser learning rate
n-steps	128	Rollout buffer size per env
batch-size	4	Mini-batch size for optimisation
n-epochs	4	optimisation epochs per update
gamma	0.99	Reward discount factor
gae-lambda	0.95	GAE parameter for advantage estimation
vf-coef	0.5	Value function coefficient in loss
max-grad-norm	0.5	Gradient clipping threshold
ent-coef	0.01	Entropy coefficient for exploration
clip-range	0.1	PPO surrogate objective clipping
n-envs	4	Parallel training environments
n-eval-envs	4	Parallel evaluation environments
n-vals	4	Episodes per evaluation check
eval-interval	10,000	Timesteps between evaluations
total-timesteps	600,000	Total training budget per robot

Table 4: Hyperparameters for the PPO Control Optimiser.

orientation, position, and velocity of the centre of mass of the object with respect to the robot.

Action: at each time step, the controller outputs an action vector. Every element of the vector corresponds to an actuator voxel in the body and provides an action value $u \in [0.6, 1.6]$, which corresponds to a gradual deformation of that actuator voxel to u times its rest length.

Reward: each task in the suite has a pre-specified reward function which measures the performance of the controller on the task. Using this function, the controller is fed a reward at each step. For detailed reward functions, see Appendix B.2.

B.2 Reward functions

Let p^o be the position vector for the centre of mass of object o , with components p_x^o and p_y^o . Let θ^o be a scalar describing the orientation of object o . The methods for calculating these quantities can be found in the appendix of Bhatia et al. (2021). Finally, let g be a vector describing a goal position in space, with components g_x and g_y . Table 5 displays the reward functions and their interpretations for each task we consider.

B.3 Extended PPO Budget Results

The figures in this section provide a detailed view of the training dynamics for our PPO controllers over extended durations. Figures 4 and 5 show that across multiple seeds and task variants, mean evaluation rewards consistently reach a stable plateau between 400,000 and 600,000 timesteps. These results demonstrate that extending the budget to 1,000,000 or 1,500,000 timesteps yields negligible performance gains, justifying the use of a 600,000-

step budget for the primary co-design experiments.

B.4 Cross-Task Transfer Visualizations

Figure 6 provides the full reward curves for the cross-task evaluations discussed in Section 5.2. These curves illustrate reward stability and asymptotic performance of various body plans over the 600,000-timestep budget across different environments. Collectively, they provide further evidence of latent capability and morphological specialisation across the EvoGym suite.

C Additional safety-aware co-design results

The following summarises results for task pairs not discussed in the main body. Full numerical results are provided in Table 2.

Walker-v0 (safe) vs. Carrier-v0 (unsafe). The regular GA on Walker-v0 achieved a best reward of 9.52, and the same body obtained 3.36 when cross-trained on Carrier-v0. Under the joint objective, the best body achieved 9.31 on Walker-v0 and -0.81 on Carrier-v0. Carrying performance was strongly suppressed while walking performance remained high. The evolved body developed rigid hindquarters, forcing it into a galloping gait that caused the box to be thrown off quickly.

Carrier-v0 (safe) vs. Walker-v0 (unsafe). The joint GA produced a best body with a reward of 2.93 on Carrier-v0 and 0.12 on Walker-v0. The body could move horizontally only while the box remained inside it, indicating that the evolved morphology retained some limited carrying ability while suppressing general walking capability.

Task	Reward function	Description
Walker-v0	Δp_x^r	Rewarded for moving to the right
Balancer-v0	$-\Delta g_x - p_x^r - \Delta g_y - p_y^r $	Penalised for moving away from goal position
Pusher-v0	$0.5\Delta p_x^r + 0.75\Delta p_x^b - \Delta p_x^r - p_x^b $	Rewarded for moving robot and box to the right, penalised for separation between box and robot
Carrier-v0	$0.5\Delta p_x^r + 0.5\Delta p_x^b - \begin{cases} 0 & p_y^b \geq t_y \\ 10\Delta p_y^b & \text{otherwise} \end{cases}$	Rewarded for robot and box moving to the right, penalised for dropping box below threshold height t_y
Flipper-v0	$\Delta\theta^r$	Rewarded for rotating
Jumper-v0	$10\Delta p_y^r - 5 \Delta p_x^r $	Rewarded for gaining height and penalised for horizontal motion

Table 5: Reward functions for the tasks considered in this paper.

Balancer-v0 (safe) vs. Walker-v0 (unsafe). Due to the extreme reward scale mismatch between tasks, the unsafe-task penalty was rescaled to be comparable to Balancer rewards. The regular GA on Balancer-v0 achieved 0.096, while the same body reached 4.41 when cross-trained on Walker-v0. The joint GA achieved 0.085 on Balancer-v0 and -0.245 on Walker-v0. The joint body evolved fewer limbs and fewer actuator voxels than the single-task balancer.

Walker-v0 (safe) vs. Jumper-v0 (unsafe). The regular GA on Walker-v0 achieved 9.54 while the regular GA on Jumper-v0 achieved 0.69, yielding $c = 13.86$. The joint GA successfully preserved walking performance (9.45) while substantially reducing jumping capability (0.32), suggesting that jumping and walking rely on sufficiently distinct morphological features.

D Unnormalised rewards for the scaling-parameter experiment

Figure 7 shows the unnormalised rewards for the scaling-parameter experiment discussed in Section 4.4. The safe task is Flipper-v0 and the unsafe task is Walker-v0. Above the graph is a visualisation of the bodies trained for different values of c , ordered from lowest c (left) to highest c (right).

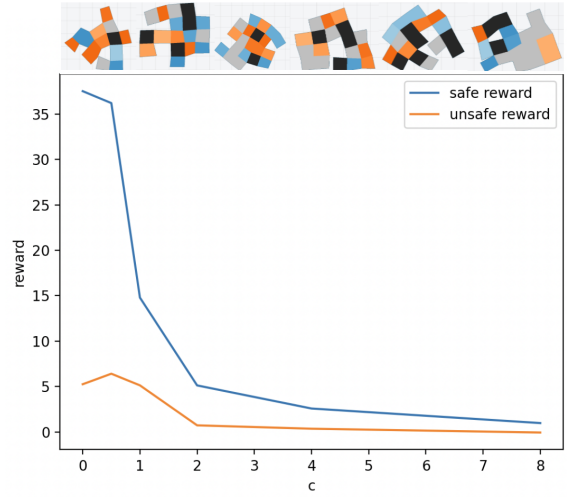
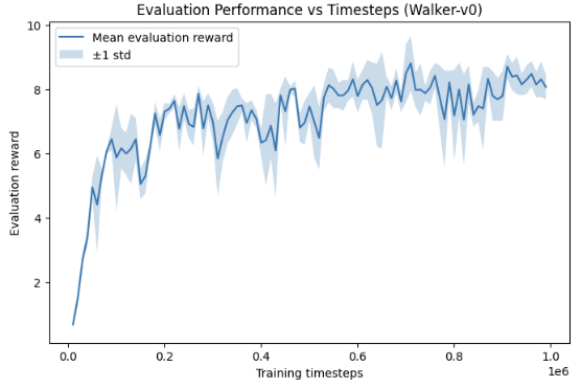
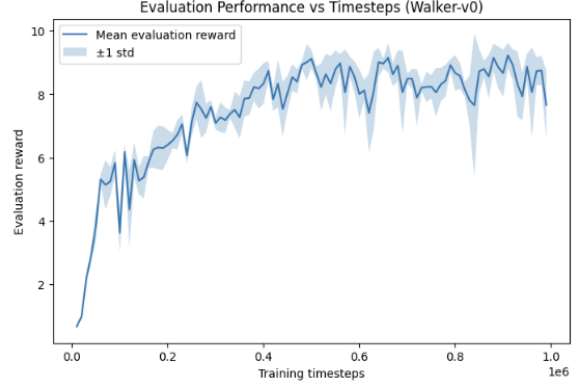


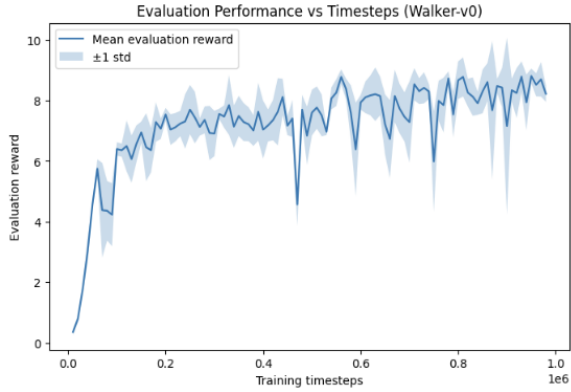
Figure 7: Unnormalised rewards for Flipper-v0 (safe) vs. Walker-v0 (unsafe) for different values of c .



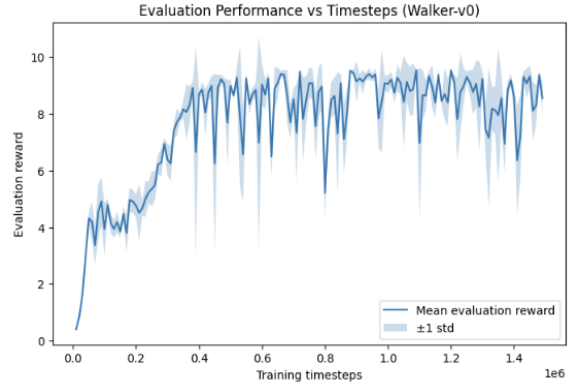
(a) Walker-v0 (1M Steps) - Run 1



(b) Walker-v0 (1M Steps) - Run 2

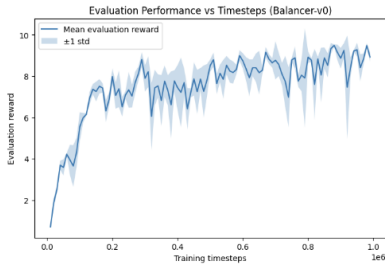


(c) Walker-v0 (1M Steps) - Run 3

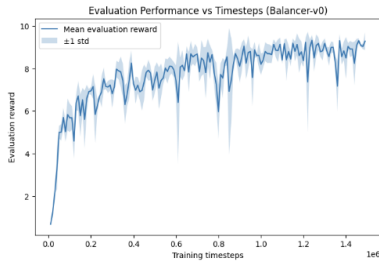


(d) Walker-v0 (1.5M Steps)

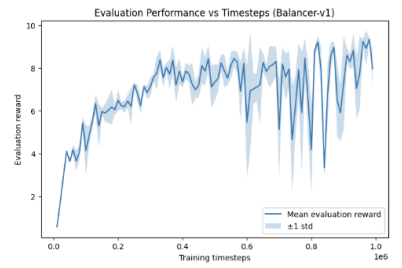
Figure 4: PPO training curves for Walker-v0 across multiple seeds and budgets. Performance consistently plateaus between 400k and 600k timesteps.



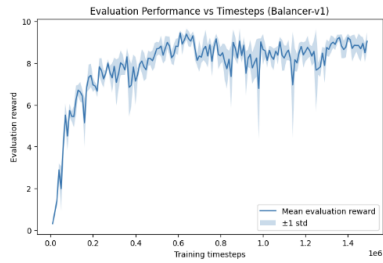
(a) Balancer-v0 (1M)



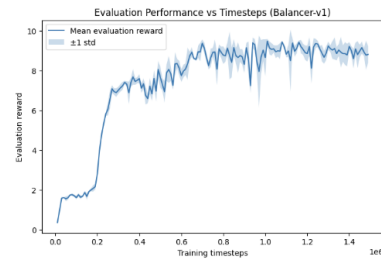
(b) Balancer-v0 (1.5M)



(c) Balancer-v1 (1M)

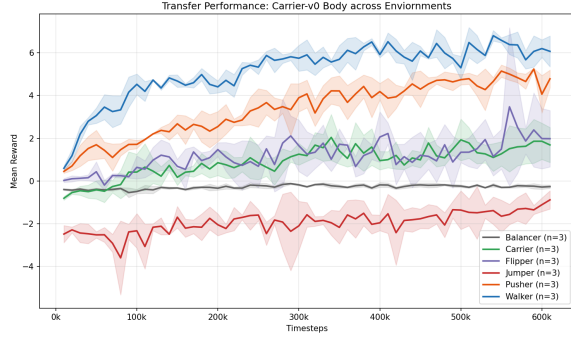


(d) Balancer-v1 (1.5M, Run 1)

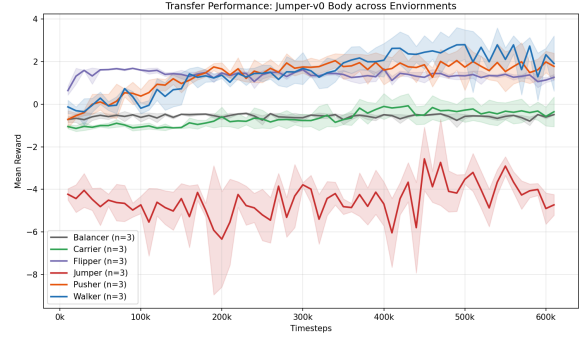


(e) Balancer-v1 (1.5M, Run 2)

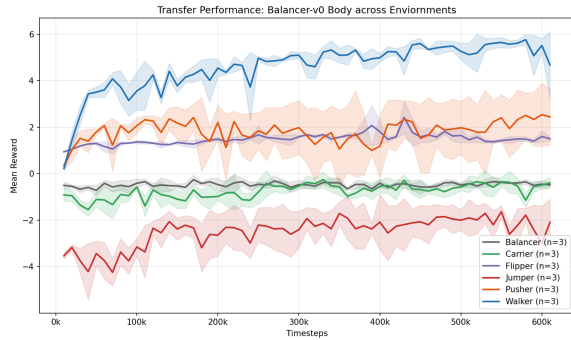
Figure 5: PPO training curves for Balancer-v0 and Balancer-v1 variants, demonstrating comparable plateau behaviour to the locomotion tasks.



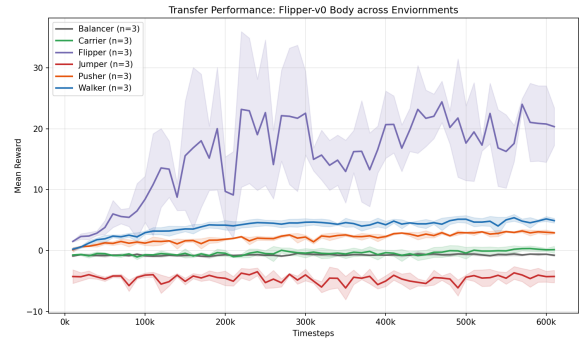
(a) Carrier-v0 Morphology



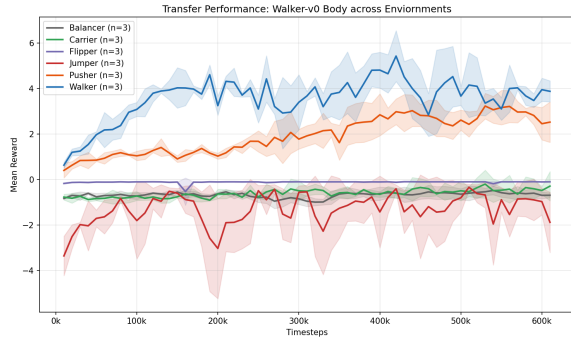
(b) Jumper-v0 Morphology



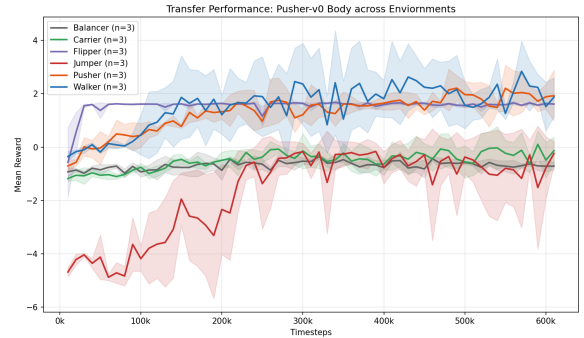
(c) Balancer-v0 Morphology



(d) Flipper-v0 Morphology



(e) Walker-v0 Morphology



(f) Pusher-v0 Morphology

Figure 6: Cross-task training performance across six EvoGym environments over a 600k timestep budget. (a) demonstrates high latent transfer for the Carrier-v0 morphology, which achieves competitive rewards on several non-native tasks. (b) illustrates the convergence failure for Jumper-v0 due to high reward variance on its native task. (c–f) show the specialization and transfer dynamics for the Balancer, Flipper, Walker, and Pusher body plans across the full evaluation suite.